

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Комп'ютерна графіка та обробка зображень»

Робота №2

Тема «Створення шейдерів»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Туленков А.В.

2025

ЗАВДАННЯ

Завдання на роботу зі сторінки 16:

1. Змініть фігуру на будь яку іншу нестандартну.
2. Змініть колір фігури та розташування.
3. Наведіть коментарі до коду.
4. Внесіть індивідуальні зміни до коду

РЕЗУЛЬТАТ

```
#include <stdio.h>
#include <string.h>
#include <GL/glew.h>
#include <GLFW/glfw3.h> // Потрібна 'h' для коректного включення

// Константи для розмірів вікна (не впливають на розмір буфера кадру,
// але використовуються при створенні)
const GLint WIDTH = 800, HEIGHT = 800;
// Ідентифікатори OpenGL об'єктів
GLuint VAO, VBO, shader;

/*
 *
 * =====
 * ШЕЙДЕРИ GLSL
 * =====
 */

// Вершинний шейдер
static const char *vShader = "
#version 330
\n\
// 'pos' - це вхідний атрибут (вершинна позиція), прив'язаний до
location 0 (викликається в glVertexAttribPointer)
layout(location = 0) in vec3 pos;
\n\
void main()
{
    // 2. Зміна розташування та розміру:
    // Оригінальні координати вершин (від -1.0 до 1.0)
    // - Спочатку зменшуємо фігуру в 0.3 рази (індивідуальна зміна)
    // - Потім зміщуємо фігуру:
```

```

        // - +0.6f до x, щоб перемістити її вправо (верхній правий
квадрант)
        // - +0.6f до y, щоб перемістити її вгору (верхній правий
квадрант)
        // Результат: фігура 0.3x від оригінального розміру, центрована на
(0.6, 0.6)
        gl_Position = vec4(0.3 * pos.x + 0.6f, 0.3 * pos.y + 0.6f, pos.z,
1.0);
    }
    ";

// Фрагментний шейдер
static const char *fShader = "
#version 330
\n\
// 'colour' - вихідний колір фрагмента (пікселя)
out vec4 colour;
\n\
void main()
{
    // 2. Зміна кольору фігури:
    // Встановлюємо фіолетовий колір: R=0.8, G=0.1, B=0.9, A=1.0
(індивідуальна зміна)
    colour = vec4(0.8f, 0.1f, 0.9f, 1.0f);
}
";

/*
*
=====
* ФУНКЦІЇ ДЛЯ ГЕНЕРАЦІЇ ГЕОМЕТРІЇ
*
=====
*/

/**
* @brief Створює буфери вершин (VAO та VBO) для фігури.
* * 1. Зміна фігури: Змінено на **Діамант** (Diamond), який
складається з 4-х вершин,
* але рендериться двома трикутниками (GL_TRIANGLES) з 6-ма вершинами.
*/
void CreateTriangle()
{
    // 1. Координати вершин для Діаманта (ромба), центрованого на (0,
0)
    GLfloat vertices[] = {
        // Трикутник 1 (Верхній)
        0.0f, 1.0f, 0.0f,    // Top point (A)
        -1.0f, 0.0f, 0.0f,   // Left point (B)
        1.0f, 0.0f, 0.0f,    // Right point (C)

        // Трикутник 2 (Нижній)
        -1.0f, 0.0f, 0.0f,   // Left point (B)
        0.0f, -1.0f, 0.0f,   // Bottom point (D)
        1.0f, 0.0f, 0.0f     // Right point (C)
    };
};

```

```

// 4. Ініціалізація та налаштування VAO (Vertex Array Object)
// VAO зберігає посилання на VBO та конфігурацію атрибутів вершин
glGenVertexArrays(1, &VAO);
glBindVertexArray(VAO);

// Ініціалізація та налаштування VBO (Vertex Buffer Object)
glGenBuffers(1, &VBO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);

// Копіюємо дані вершин у буфер VBO
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

// Вказуємо, як інтерпретувати дані у VBO:
// 0: індекс атрибута (layout(location = 0) у шейдері)
// 3: кількість компонент на вершину (x, y, z)
// GL_FLOAT: тип даних
// GL_FALSE: не нормалізувати
// 0: крок між вершинами (0 означає щільну упаковку)
// 0: зміщення до першого компонента
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);

// Активуємо атрибут вершин з location 0
glEnableVertexAttribArray(0);

// Відв'язуємо VBO та VAO
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
}

/*
*
=====
==
* ФУНКЦІЇ ДЛЯ КОМПІЛЯЦІЇ ШЕЙДЕРІВ
*
=====
==
*/

/**
* @brief Додає та компілює окремий шейдер (вершинний або фрагментний)
до програми.
*/
void AddShader(GLuint theProgram, const char *shaderCode, GLenum
shaderType) // Виправлено ім'я функції: AddShader
{
    GLuint theShader = glCreateShader(shaderType);

    // 4. Виправлення синтаксису: оголошення масиву `theCode` та
`codeLength`
    const GLchar *theCode[1];
    theCode[0] = shaderCode;
    GLint codeLength[1];
    codeLength[0] = strlen(shaderCode);

    // Встановлюємо джерело шейдера

```

```

glShaderSource(theShader, 1, theCode, codeLength);

// Компілюємо шейдер
glCompileShader(theShader);

// Перевірка помилок компіляції
GLint result = 0;
GLchar eLog[1024] = {0};
glGetShaderiv(theShader, GL_COMPILE_STATUS, &result);
if (!result)
{
    glGetShaderInfoLog(theShader, 1024, NULL, eLog);
    // Виводимо тип шейдера для кращої діагностики
    fprintf(stderr, "Error compiling the %s shader: '%s'\n",
            (shaderType == GL_VERTEX_SHADER ? "vertex" :
"fragment"), eLog);
    return;
}

// Прикріплюємо шейдер до програми
glAttachShader(theProgram, theShader);
}

/**
 * @brief Створює, компілює та лінкує шейдерну програму.
 */
void CompileShaders() // виправлено ім'я функції: CompileShaders
{
    // Створення програми шейдерів
    shader = glCreateProgram();
    if (!shader)
    {
        printf("Failed to create shader program\n");
        return;
    }

    // Додаємо вершинний та фрагментний шейдери
    AddShader(shader, vShader, GL_VERTEX_SHADER); // виправлено ім'я
функції
    AddShader(shader, fShader, GL_FRAGMENT_SHADER); // виправлено ім'я
функції

    // Лінування (з'єднання) програми
    GLint result = 0;
    GLchar eLog[1024] = {0};
    glLinkProgram(shader);

    // Перевірка помилок лінування
    glGetProgramiv(shader, GL_LINK_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error linking program: '%s'\n", eLog);
        return;
    }

    // Валідація програми (перевірка придатності для поточного стану
OpenGL)

```

```

    glValidateProgram(shader);
    glGetProgramiv(shader, GL_VALIDATE_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error validating program: '%s'\n", eLog);
        // Примітка: Валідація може не вдатися навіть при успішному
лінкуванні,
        // але програма все ще може працювати на деяких конфігураціях.
        return;
    }
}

/*
*
=====
==
* ОСНОВНА ФУНКЦІЯ
*
=====
==
*/

int main()
{
    // Ініціалізація GLFW
    if (!glfwInit())
    {
        printf("GLFW failed to start :(\n"); // Додано \n для кращого
форматування
        glfwTerminate();
        return 1; // Виправлено return 1 на return 1;
    }

    // Налаштування параметрів вікна OpenGL
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    // Використовуємо Core Profile (без застарілих функцій)
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    // Забезпечуємо пряму сумісність (потрібно для Mac OS)
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

    // Створення вікна
    GLFWwindow *mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Custom
OpenGL Diamond", NULL, NULL); // Індивідуальна зміна: назва вікна
    if (!mainWindow)
    {
        printf("GLFW window creation failed :(\n");
        glfwTerminate();
        return 1;
    }

    // Отримання розмірів буфера кадру (може відрізнятись від розміру
вікна на HiDPI дисплеях)
    int bufferWidth, bufferHeight;
    glfwGetFramebufferSize(mainWindow, &bufferWidth, &bufferHeight);

    // Встановлення поточного контексту OpenGL на створене вікно

```

```

glfwMakeContextCurrent(mainWindow);

// Ініціалізація GLEW (потрібна для доступу до функцій OpenGL)
glewExperimental = GL_TRUE; // Дозволяє використовувати сучасні
функції OpenGL
if (glewInit() != GLEW_OK)
{
    printf("GLEW failed to start >_<\n");
    glfwDestroyWindow(mainWindow);
    glfwTerminate();
    return 1;
}

// Налаштування області відображення OpenGL
glViewport(0, 0, bufferWidth, bufferHeight);

// Створюємо геометрію та компілюємо шейдери
CreateTriangle();
CompileShaders();

// Основний цикл рендерингу
while (!glfwWindowShouldClose(mainWindow))
{
    // Обробка подій (введення з клавіатури/миші)
    glfwPollEvents();

    // 4. Індивідуальна зміна: Зміна кольору фону на **морську
хвилю**
    glClearColor(0.1f, 0.7f, 0.6f, 1.0f);
    // Очищення буфера кольору
    glClear(GL_COLOR_BUFFER_BIT);

    /*
    *
    =====
    ==
    * ВІДБРАЖЕННЯ ФІГУРИ
    *
    =====
    ==
    */

    // Використовуємо нашу шейдерну програму
    glUseProgram(shader);

    // Прив'язуємо VAO, яке містить дані про фігуру (Діамант)
    glBindVertexArray(VAO);

    // Малюємо примітив:
    // GL_TRIANGLES: малюємо фігуру як набір окремих трикутників
    // 0: індекс першої вершини
    // 6: загальна кількість вершин для Діаманта (2 трикутники * 3
вершини/трикутник)
    glDrawArrays(GL_TRIANGLES, 0, 6);

    // Відв'язуємо VAO та шейдерну програму
    glBindVertexArray(0);
    glUseProgram(0);

```

```
        // Обмін буферами (передача нового кадру на екран)
        glfwSwapBuffers(mainWindow);
    }

    // Очищення ресурсів після завершення циклу
    glDeleteProgram(shader);
    glDeleteVertexArrays(1, &VAO);
    glDeleteBuffers(1, &VBO);

    // Завершення роботи GLFW
    glfwTerminate();
    return 0;
}
```